Next.js Mastery Series — Topic 8 of N

Middleware

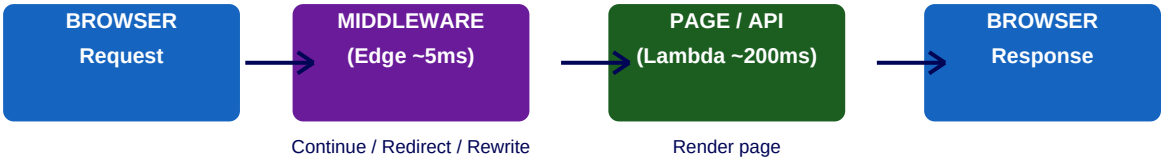
Edge Runtime · Auth · RBAC · i18n · Rate Limiting · Redirect vs Rewrite

Akshay Chavhan | Target: Syngenta India + All Companies | April 2026

■ BEGINNER LEVEL

What is Middleware?

Middleware is a function in **middleware.ts** at the project root that runs **before every request reaches your page or API route**. Think of it as a security guard — every request must pass through it first. It can let requests through, redirect them, or rewrite them.



Where to Create middleware.ts

```
my-app/
■■■ src/
■   ■■■ middleware.ts    ← HERE (if using src/ folder)
■   ■■■ app/
OR:
■■■ middleware.ts        ← HERE (if NOT using src/)
■■■ app/

// Rules:
// ■ ONE middleware.ts file per project only
// ■ Must be at project ROOT or inside src/ root
// ■ Cannot be inside app/ or pages/
// ■ Cannot have multiple middleware files
```

Most Basic Middleware

```
// middleware.ts
import { NextRequest, NextResponse } from 'next/server';

export function middleware(request: NextRequest) {
  console.log("Request to:", request.nextUrl.pathname);

  // Continue to the requested page normally
  return NextResponse.next();
}

// Every request in your entire app goes through this ■
```

The 3 Things Middleware Can Return

Return	What Happens	Use Case
NextResponse.next()	Request continues normally	Logging, adding headers, setting cookies
NextResponse.redirect(url)	User sent to different URL (URL changes)	Auth redirect to /login, old URL redirect
NextResponse.rewrite(url)	Different page renders, URL stays SAME	A/B testing, maintenance mode, i18n

Common Use Cases

Use Case	What It Does
Authentication	Check token/cookie, redirect to /login if not logged in
Authorization (RBAC)	Check user role, redirect to /unauthorized if wrong role
i18n Routing	Detect language from headers, rewrite to /en, /hi, /mr
Rate Limiting	Count requests per IP, return 429 if too many
A/B Testing	Randomly rewrite to /page-a or /page-b, same URL
Maintenance Mode	Rewrite all traffic to /maintenance page
Logging	Log every request before it reaches the page
Header Injection	Add x-user-id header so pages can read user without DB

■ INTERMEDIATE LEVEL

Edge Runtime vs Node.js Runtime

This is the most important thing to understand about middleware. It does NOT run on your Node.js server — it runs on the **Edge Runtime** at Vercel's CDN, close to the user. Ultra-fast but limited APIs.

EDGE RUNTIME (Middleware)

```
Where: CDN edge servers (30+ cities)
Cold start: ~5ms
For Syngenta user in Pune:
  runs on Mumbai edge server

CAN use:
  Web APIs (fetch, URL, Headers)
  Cookies, Request/Response
  jose (JWT verify)
  Upstash Redis

CANNOT use:
  Prisma / DB queries
  Node.js fs, path, crypto
  Heavy npm packages
```

NODE.JS RUNTIME (Pages, APIs)

```
Where: Lambda functions (server)
Cold start: ~200ms
For Syngenta user in Pune:
  Lambda might be in Mumbai

CAN use:
  Prisma / DB queries
  Node.js built-ins (fs, path)
  Any npm package
  Server Actions

CANNOT use:
  Edge restrictions
```

The matcher — Control Which Routes Get Middleware

By default middleware runs on EVERY request including static files and images. Use **matcher** to limit it to specific routes only — this is critical for performance.

```
// middleware.ts

export function middleware(request: NextRequest) {
  // your logic
  return NextResponse.next();
}

// matcher — WHICH routes trigger this middleware
export const config = {
  matcher: [
    // Run everywhere EXCEPT static files, images, favicon
    '/((?!_next/static|_next/image|favicon.ico|public).*)',

    // OR — run on specific routes only:
    '/dashboard/:path*',    // /dashboard and all children
    '/admin/:path*',        // /admin and all children
    '/api/:path*',          // all API routes
  ]
};
```

Auth Protection — Most Common Pattern

```
// middleware.ts
import { NextRequest, NextResponse } from 'next/server';

export function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // Get auth token from cookie
  const token = request.cookies.get('auth-token')?.value;

  // Protected routes
  const isProtected = pathname.startsWith("/dashboard") ||
                      pathname.startsWith("/admin") ||
                      pathname.startsWith("/profile");

  // If protected + no token → redirect to login
  if (isProtected && !token) {
    const loginUrl = new URL("/login", request.url);
    loginUrl.searchParams.set("redirect", pathname);
    return NextResponse.redirect(loginUrl);
  }

  // If logged in + on login page → redirect to dashboard
  if (pathname === "/login" && token) {
    return NextResponse.redirect(new URL("/dashboard", request.url));
  }

  return NextResponse.next();
}

export const config = {
  matcher: ['/((?!_next/static|_next/image|favicon.ico).*)'],
};
```

Redirect vs Rewrite — Side by Side

```
// REDIRECT — URL changes in browser bar
return NextResponse.redirect(new URL('/login', request.url));
// User visits /dashboard (no auth)
// Browser URL changes to: /login
// User KNOWS they were redirected

// REWRITE — URL stays same, different content renders
return NextResponse.rewrite(new URL('/maintenance', request.url));
// User visits /dashboard
// Browser URL STAYS: /dashboard
// But /maintenance page content is shown
// User has NO idea the rewrite happened

// Real use cases:
// REDIRECT → auth redirect, old URL cleanup
// REWRITE  → A/B testing, maintenance mode, i18n routing
```

Reading and Setting Cookies

```
export function middleware(request: NextRequest) {
  // READ cookie from request
  const token  = request.cookies.get('auth-token')?.value;
  const theme  = request.cookies.get('theme')?.value;

  const response = NextResponse.next();

  // SET cookie on response
  response.cookies.set('theme', 'dark', {
    httpOnly: true,        // not accessible by browser JS
    secure: true,          // HTTPS only
    sameSite: 'lax',       // CSRF protection
    maxAge: 60 * 60 * 24   // 1 day in seconds
  });

  // DELETE cookie
  response.cookies.delete('old-session');

  return response;
}
```

Header Injection — Pass Data to Pages

```
// Add user info to headers so Server Components can read it
// without making a DB call in every page

export function middleware(request: NextRequest) {
  const requestHeaders = new Headers(request.headers);

  // Add custom data to request headers
  requestHeaders.set('x-user-id',    'user_123');
  requestHeaders.set('x-user-role',   'admin');
  requestHeaders.set('x-pathname',    request.nextUrl.pathname);

  return NextResponse.next({
    request: { headers: requestHeaders }
  });
}

// In any Server Component/layout, read with:
import { headers } from 'next/headers';
const headersList = headers();
const userId = headersList.get('x-user-id');
const role   = headersList.get('x-user-role');
```

■ EXPERT LEVEL

JWT Verification at the Edge

The correct pattern for auth middleware — verify JWT token at the edge (no DB call needed). Pass user info via headers to pages that need full access.

```
import { jwtVerify } from 'jose'; // lightweight, Edge-compatible

const SECRET = new TextEncoder().encode(process.env.JWT_SECRET!);

export async function middleware(request: NextRequest) {
  const token = request.cookies.get('auth-token')?.value;

  if (!token) {
    return NextResponse.redirect(new URL("/login", request.url));
  }

  try {
    // Verify JWT — ~1ms, no DB needed!
    const { payload } = await jwtVerify(token, SECRET);

    // Inject user data into headers for pages to read
    const headers = new Headers(request.headers);
    headers.set("x-user-id",   payload.sub as string);
    headers.set("x-user-role", payload.role as string);

    return NextResponse.next({ request: { headers } });

  } catch {
    // Token invalid or expired
    return NextResponse.redirect(new URL("/login", request.url));
  }
}

export const config = {
  matcher: ['/dashboard/:path*', '/admin/:path*'],
};
```

Role-Based Access Control (RBAC)

```
import { jwtVerify } from 'jose';

// Define which roles can access which routes
const ROUTE_ROLES: Record<string, string[]> = {
  '/admin':      ['admin'],
  '/reports':    ['admin', 'manager'],
  '/dashboard':  ['admin', 'manager', 'user'],
};

export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;
  const token = request.cookies.get('auth-token')?.value;

  if (!token) return NextResponse.redirect(new URL("/login", request.url));

  try {
    const { payload } = await jwtVerify(token, SECRET);
    const userRole = payload.role as string;

    // Find required roles for this route
    const required = Object.entries(ROUTE_ROLES)
      .find(([route]) => pathname.startsWith(route))?.[1];

    if (required && !required.includes(userRole)) {
      // Logged in but wrong role
      return NextResponse.redirect(new URL("/unauthorized", request.url));
    }

    return NextResponse.next();
  } catch {
    return NextResponse.redirect(new URL("/login", request.url));
  }
}
```

i18n — Auto Language Detection

```
const LOCALES  = ['en', 'hi', 'mr']; // English, Hindi, Marathi
const DEFAULT  = 'en';

export function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // Already has locale prefix? Continue
  const hasLocale = LOCALES.some(
    locale => pathname.startsWith(`/${locale}/`)
              || pathname === `/${locale}`
  );
  if (hasLocale) return NextResponse.next();

  // Detect from browser Accept-Language header
  const acceptLang = request.headers.get('accept-language') ?? '';
  const detected = LOCALES.find(l => acceptLang.includes(l)) ?? DEFAULT;

  // Rewrite — URL stays same, localized content renders
  const newUrl = new URL(`/${detected}${pathname}`, request.url);
  return NextResponse.rewrite(newUrl);
}
```

Rate Limiting with Upstash Redis

```
import { Ratelimit } from '@upstash/ratelimit'; // Edge compatible
import { Redis }     from '@upstash/redis';

// 10 requests per 10 seconds per IP
const ratelimit = new Ratelimit({
  redis:   Redis.fromEnv(),
  limiter: Ratelimit.slidingWindow(10, "10 s"),
});

export async function middleware(request: NextRequest) {
  const ip = request.ip ?? '127.0.0.1';
  const { success, limit, remaining } = await ratelimit.limit(ip);

  if (!success) {
    return NextResponse.json(
      { error: 'Too many requests' },
      { status: 429, headers: {
        'X-RateLimit-Limit':     limit.toString(),
        'X-RateLimit-Remaining': remaining.toString(),
      }}
    );
  }

  return NextResponse.next();
}

export const config = {
  matcher: ['/api/:path*'],  // rate limit API routes only
};
```

■■ Critical Gotchas

■■ Gotcha 1: Middleware runs on EVERY request — keep it fast. Target under 10ms. NEVER do DB queries in middleware. If you need user data, verify JWT (1ms) and read what is inside the token. DB queries in middleware = massive performance hit at scale.

■■ Gotcha 2: Cannot use Prisma in middleware. Prisma needs Node.js runtime. Middleware runs on Edge Runtime (V8 isolate only). Use jose for JWT verification, Upstash Redis for KV lookups. These are Edge-compatible.

■■ Gotcha 3: Only ONE middleware.ts per project. You cannot have multiple files. Use the matcher config and conditional logic inside the single middleware to handle different route groups.

■■ Gotcha 4: Matcher must be statically analyzable. You cannot use runtime variables, dynamic imports, or function calls to generate the matcher array. Patterns must be literal strings defined at module level.

■ Best Practice: Always exclude static assets from middleware with matcher. Add the (?!_next/static|_next/image|favicon.ico) pattern. Middleware running on image requests adds unnecessary latency and cost.

■ Best Practice: Use header injection (x-user-id, x-user-role) to pass middleware-verified data to Server Components. This avoids repeating auth checks or DB calls in every layout and page that needs user info.

■ INTERVIEW Q&A WITH DETAILED ANSWERS

Beginner Level

BEGINNER

Q1. What is Middleware in Next.js?

Answer: Middleware is a function in middleware.ts at the project root that runs before every request reaches your page or API route. It can continue the request, redirect the user, or rewrite the URL. Common uses: authentication checks, redirects, i18n routing, rate limiting, A/B testing, logging.

BEGINNER

Q2. Where does middleware.ts go and how many can you have?

Answer: Exactly ONE middleware.ts per project — either at the root (no src/) or inside src/ if you use that structure. Cannot go inside app/ or pages/. Use the matcher export config to control which routes trigger it. Use conditional logic inside the one file to handle different route groups.

BEGINNER

Q3. What are the 3 things middleware can return?

Answer: NextResponse.next() — let request continue normally. Used for logging, header injection, setting cookies. NextResponse.redirect(url) — send user to different URL, browser URL changes. Used for auth redirects. NextResponse.rewrite(url) — render different content but keep original URL in browser. Used for A/B testing, maintenance mode.

Intermediate Level

INTERMEDIATE

Q4. Where does middleware run — server or edge? Why does it matter?

Answer: Middleware runs on the Edge Runtime — Vercel CDN edge servers in 30+ cities worldwide, NOT on your main Node.js Lambda. Cold start is ~5ms vs ~200ms for Lambda. For a Syngenta user in Pune, middleware runs on the nearest Mumbai edge server. This is why auth redirects and i18n rewrites feel instant — no round-trip to a distant server.

INTERMEDIATE

Q5. What is the difference between redirect and rewrite in middleware?

Answer: Redirect changes the URL in the browser bar — user sees the new URL. Used for auth (redirect to /login). Rewrite keeps the original URL but renders different page content — user has NO idea the rewrite happened. Used for A/B testing (/pricing shows either /pricing-a or /pricing-b content), maintenance mode, or i18n routing where you want clean URLs.

INTERMEDIATE

Q6. Why can you NOT use Prisma in middleware?

Answer: Prisma requires Node.js runtime — it uses TCP connections, file system, and Node.js-specific APIs. Middleware runs on Edge Runtime which is a lightweight V8 isolate with only Web standard APIs. No Node.js built-ins available. Alternative: verify a JWT token (contains user data baked in, no DB needed, ~1ms) and inject user info into request headers for pages.

INTERMEDIATE

Q7. What is the matcher and why is it important?

Answer: Matcher is an exported config object that tells Next.js which routes trigger middleware. Without it, middleware runs on EVERY request including static files, images, fonts — wasting compute. Use (?!_next/static|_next/image|favicon.ico) pattern to exclude static assets. Use /dashboard/:path* to match only dashboard routes. Patterns must be statically analyzable — no runtime variables.

Expert / Syngenta-Style Questions

EXPERT

Q8. How do you implement role-based access control in middleware without a DB call?

Answer: Verify JWT token from cookies using jose (Edge-compatible). Extract role from token payload — role was baked into the token at login time, no DB needed. Maintain a static route-to-roles mapping object. For each request, find matching route pattern, check if user role is allowed. Redirect to /unauthorized if role does not match. Entire check runs in ~2ms at the edge — zero DB calls.

EXPERT

Q9. How do you implement rate limiting in Next.js middleware?

Answer: Use Upstash Redis with @upstash/ratelimit — both Edge Runtime compatible. Create Ratelimit instance with sliding window config (e.g. 10 req per 10 seconds). Extract client IP from request.ip. Call ratelimit.limit(ip) — returns success boolean and limit headers. Return 429 with X-RateLimit headers if over limit. Apply matcher to /api/:path* only so UI routes are not rate limited.

SYNGENTA

Q10. Design the complete middleware for Syngenta farmer portal.

Answer: JWT verification for /dashboard/*, /admin/*, /reports/* — redirect to /login with ?redirect= param if invalid. RBAC: admins access /admin, managers access /reports, all roles access /dashboard. i18n: detect accept-language header, rewrite to /hi/* or /mr/* for Hindi/Marathi while keeping URL clean. Rate limiting on /api/* using Upstash — 100 req/min per IP. Header injection: add x-user-id and x-user-role so Server Components read user without DB. matcher: exclude _next/static, _next/image, favicon to avoid running on static assets.

EXPERT

Q11. How does middleware improve performance compared to checking auth in every layout?

Answer: Without middleware: every layout and page runs auth check — multiple DB calls or cookie reads per request. With middleware: auth check runs ONCE at the edge before the request even reaches Next.js server. Invalid requests are rejected at the CDN (5ms) — they never reach your Lambda or DB. Valid requests get user info injected into headers — pages read headers instantly without repeating auth logic. This reduces Lambda invocations, DB connections, and perceived latency simultaneously.